Introduction to Software Construction

SE 411, V3
Fall 2026-27

Contents

In this course, we focus on the skills needed to build high-quality software.

1. **The construction landscape**
2. **The core challenge: Managing complexity** (writing code humans can read).
3. **Under the hood: Parsers and compilers** (how machines read code).
4. **The construction environment:** Using **Git** (history) and **JUnit** (verification) to manage the chaos.

Course web site:
https://sites.google.com/view/se411

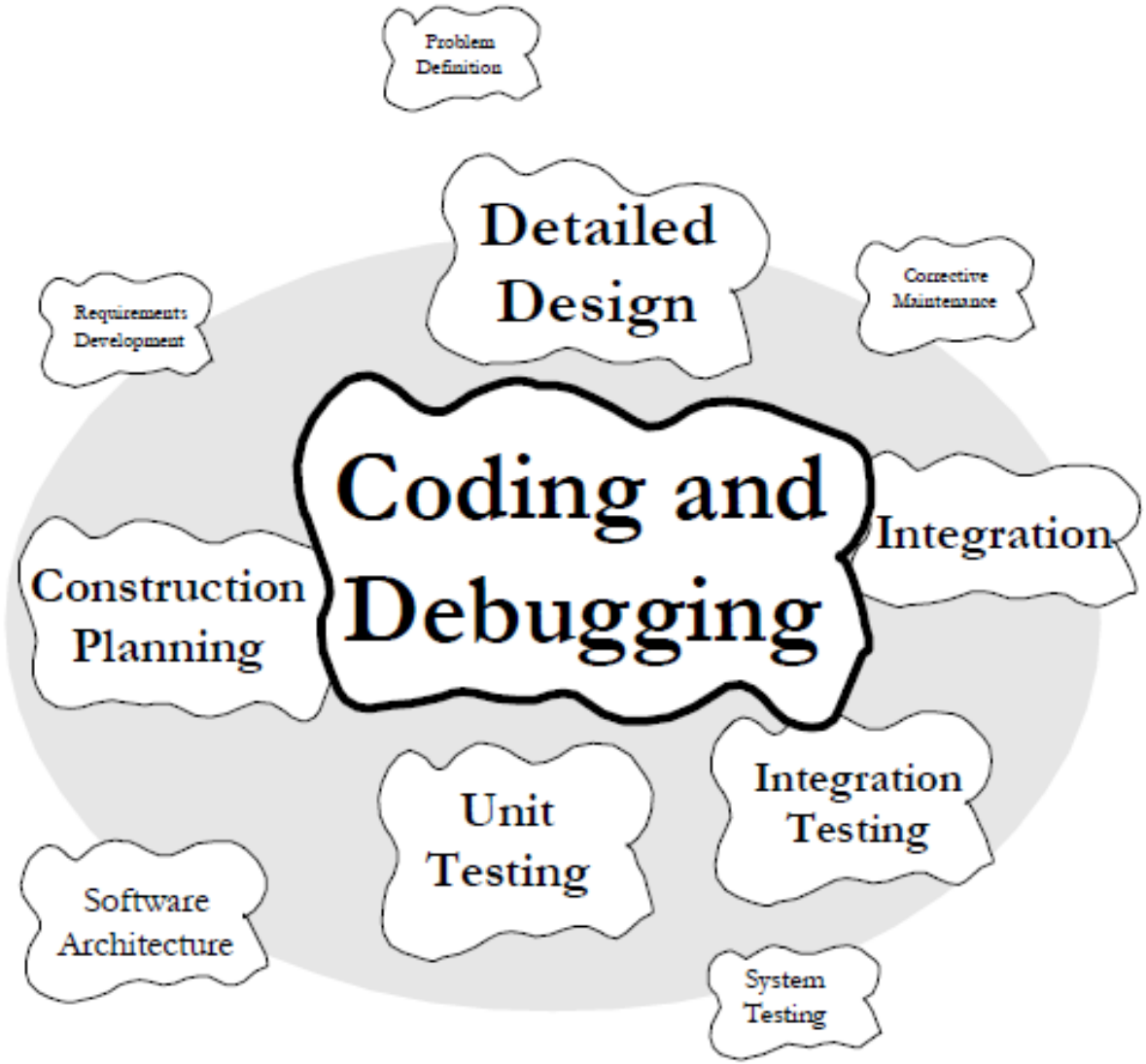
Source code repository:
https://github.com/skanderturki/se411

1. The construction landscape

What Is Software Construction?

- Software Construction is the central activity of software development. While requirements gathering happens before, and system testing happens after, **construction** is where the actual creation happens.
- It is the detailed process of:
  - **Coding:** Writing source code.
  - **Verifying:** Testing unit-by-unit to ensure correctness.
  - **Debugging:** Identifying and fixing errors.
  - **Integration:** Combining components into a working whole.
- **Key Takeaway:** Construction is the only activity guaranteed to happen on every project. It typically consumes 50-80% of project time.

Software Construction



Software Construction

- Specific tasks involved in construction:
  - Verifying that the groundwork has been laid so that construction can proceed successfully
  - Determining how your code will be tested
  - Designing and writing classes and routines
  - Creating and naming variables and named constants
  - Selecting control structures and organizing blocks of statements
  - Unit testing, integration testing, and debugging your own code
  - Reviewing other team members' low-level designs and code and having them review yours
  - Polishing code by carefully formatting and commenting it
  - Integrating software components that were created separately
  - Tuning code to make it smaller and faster

AI in the Construction Loop

- AI assistants now touch every construction activity, but guarantee none of them:
  - **Coding: generate, complete, and translate code.**
  - **Verifying: draft unit tests and suggest edge cases.**
  - **Debugging: explain errors and propose fixes.**
  - **Integration: wire components and write glue code.**
- **The catch: AI output is plausible, not guaranteed correct.**
- **Our recurring rule, Verify the AI output: never accept code you have not read and tested.**

The New Engineer’s Skill: From Typing to Verifying

- AI shifts where a developer’s time goes:
  - Less time typing boilerplate.
  - More time specifying, reviewing, and verifying.
- **You remain fully responsible for every line you submit.**
- The tool landscape:
  - Chat assistants (ChatGPT, Claude, Gemini), ask, explain, draft.
  - In-IDE (GitHub Copilot, Cursor), inline completion.
  - Agentic (Claude Code), multi-step tasks across files.

2. The core challenge: Managing Complexity

Challenges of Software Development

- Complexity: The biggest technical risk in software development is not "not knowing the syntax," but **uncontrolled complexity**. If code is too complex, we cannot understand it, modify it, or fix it safely.
  - **The Goal:** Break complicated problems into simple pieces
- Longevity and Evolution
- High User Expectations

Complexity in Software Development

- Real life problems are complex
- Requirements are out of control of developers
- Too many details in a large-scale software system
- Uncontrolled complexity may cause project failure

Managing & Minimizing Complexity

- Managing complexity is the most important technical topic in software development
- Effective techniques are required to manage and minimize complexity

➢ It is easier to comprehend several simple pieces of information than one complicated piece

➢ It is safer to focus on one bit of complexity at a time

➢ Goal of all software-design techniques is to break a complicated problem into simple pieces

Minimize Complexity in Construction

- Reduced complexity is achieved & kept by creation of code that is:
  - Simple
  - Readable
  - Understandable
  - Easy to maintain (adding feature, fixing bug, etc.)
- KISS Principle
  - Keep It Simple & Straightforward

Concrete Example: Complexity vs. Simplicity

- The following example illustrates the "KISS Principle" (Keep It Simple & Straightforward).

```
// Hard to read, deep nesting, magic numbers
public void process(int[] data) {
    if (data != null) {
        for (int i = 0; i < data.length; i++) {
            if (data[i] > 10) {
                if (data[i] % 2 == 0) {
                    System.out.println("Processing: " + data[i]);
                }
            }
        }
    }
}
```



```
// Flatter, cleaner, self-documenting
public void process(int[] data) {
    if (data == null) return; // Guard clause

    for (int number : data) {
        if (isValid(number)) {
            System.out.println("Processing: " + number);
        }
    }
}

private boolean isValid(int number) {
    return number > 10 && number % 2 == 0;
}
```

Question: In the improved code, if the definition of `isValid` changes, how many places do you need to edit compared to the "Before" code?

Maintainability

- Maintainability is a very important & desirable quality of a software system
- For maintainability
  - simple
  - readable
  - understandable code

Writing Simple Code

- Divide the code into meaningful small blocks (packages, components, classes, methods, functions, routines, etc.)
- Avoid code duplication
- Refactor continuously
- Have the fewest possible classes & methods
- Delete everything that doesn't have a purpose

Complicated code: Deep Nesting!!

```php
<?php

class MyPlugin extends Plugin {
    public function controller_action() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') { // start if ($_SERVER['REQUEST_METHOD'] === 'POST')
            $gateway = new ServiceGateway(API_KEY, API_USER, API_PASS);
            try { // start of try/catch
                $gateway->getPostData();
                $has_data = $gateway->processData();
                if ($has_data) { // start if ($has_data)
                    if ($has_data['status'] == 'Ok' && $has_data['type'] == 'transaction') { // start ($has_data['status']...
                        $transactions = Transactions::findById($has_data['txn_id']);
                        if (! $transactions) { // start if (! $transactions)
                            $my_profile = $this->get_user()->get_profile();
                            if ($my_profile) { // start if ($my_profile)
                                if (isset($has_data['payment_status']) && $has_data['payment_status'] == 'Processed') {
                                    Transaction::store($has_data);
                                    return redirect('success', 'Congratulations, you are alive!');
                                }
                                else {
                                    return redirect('error', 'Unknown payment status');
                                } // end if (isset($has_data['payment_status'])...
                            }
                            else {
                                return redirect('error', 'Unable to find profile');
                            } // end if ($my_profile)
                        }
                        else {
                            return redirect('error', 'Transaction already exists');
                        } // end if (! $transactions)
                    }
                    else {
                        return redirect('error', 'Status not okay.');
                    } // end ($has_data['status']...
                }
                else {
                    return redirect('error', 'Bad data.');
                } // end if ($has_data)
            }
            catch (Exception $e) {
                return redirect('error', $e->getMessage());
            } // end of try/catch
        }
        else {
            return redirect('error', 'You do not have access to this page');
        } // end if ($_SERVER['REQUEST_METHOD'] === 'POST')
    }
}
```

Writing Readable Code

- Readability has positive effect on
  - Comprehensibility
  - Reviewability
  - Error rate
  - Debugging
  - Modifiability
  - Development time, a consequence of all of the above
  - External quality, a consequence of all of the above

Writing Readable Code

- Write programs **for people first**, computers second
- Use good coding conventions & standards
  - naming identifiers
  - commenting
  - layout & style (aesthetics)
    - spaces, punctuation, indentation
  - etc.
- Write self-documented code

Good Names & Self-Documenting Code

- Good names are the cheapest documentation you can write.
- Name to reveal intent — a reader should know what it is without a comment.
- **Guidance:**
  - Use full, pronounceable words (customerCount, not cc).
  - Name booleans as questions (isValid, hasNext).
  - Verbs for methods (calculateTotal); nouns for classes (Invoice).
- **If a name needs a comment to explain it, improve the name.**

Comments: Explain Why, Not What

- Good code shows WHAT it does; comments explain WHY.
- Redundant comments (i++; // increment i) add noise, not value.
- **Use comments for:**
  - Intent and rationale a reader cannot infer from the code.
  - Warnings, assumptions, and non-obvious trade-offs.
- **A comment is not a fix for unclear code — rewrite the code first.**

AI & Complexity: A Double-Edged Sword

- AI can produce a lot of code very fast, including complex code.
- **This makes managing complexity MORE important, not less:**
  - AI often adds abstraction the problem does not need.
  - “It compiles and runs” is not “it is simple and maintainable.”
- Your job on AI output:
  - Apply KISS, simplify what the AI over-builds.
  - Keep it readable for the next human (probably you).
- **Reminder: uncontrolled complexity, not syntax, is the biggest risk.**

3. Under the hood: Compilers and parsers

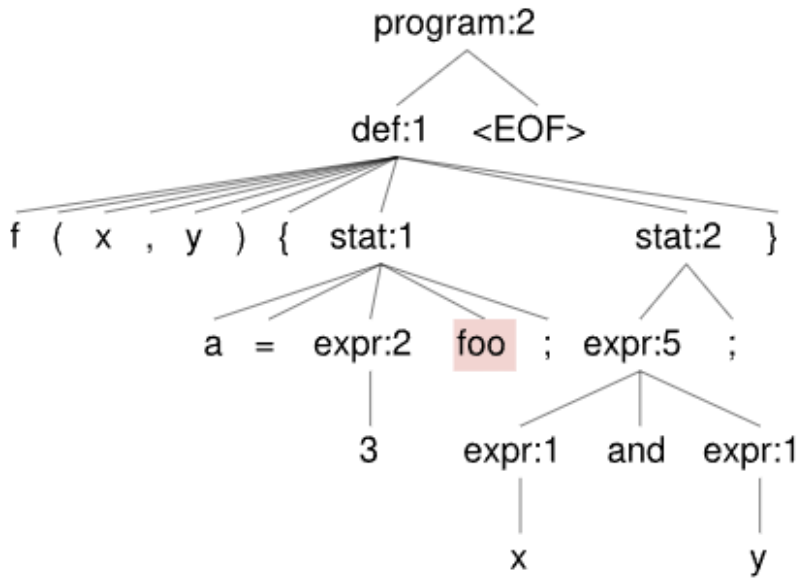
What is a parser?

A parser takes a piece of text and transforms it in an organized structure, a *parse tree*, also known as a *Abstract Syntax Tree (AST)*.

The AST is a tree representation of the code.

```
f(x,y) {
    a = 3+foo;
    x and y;
}
```

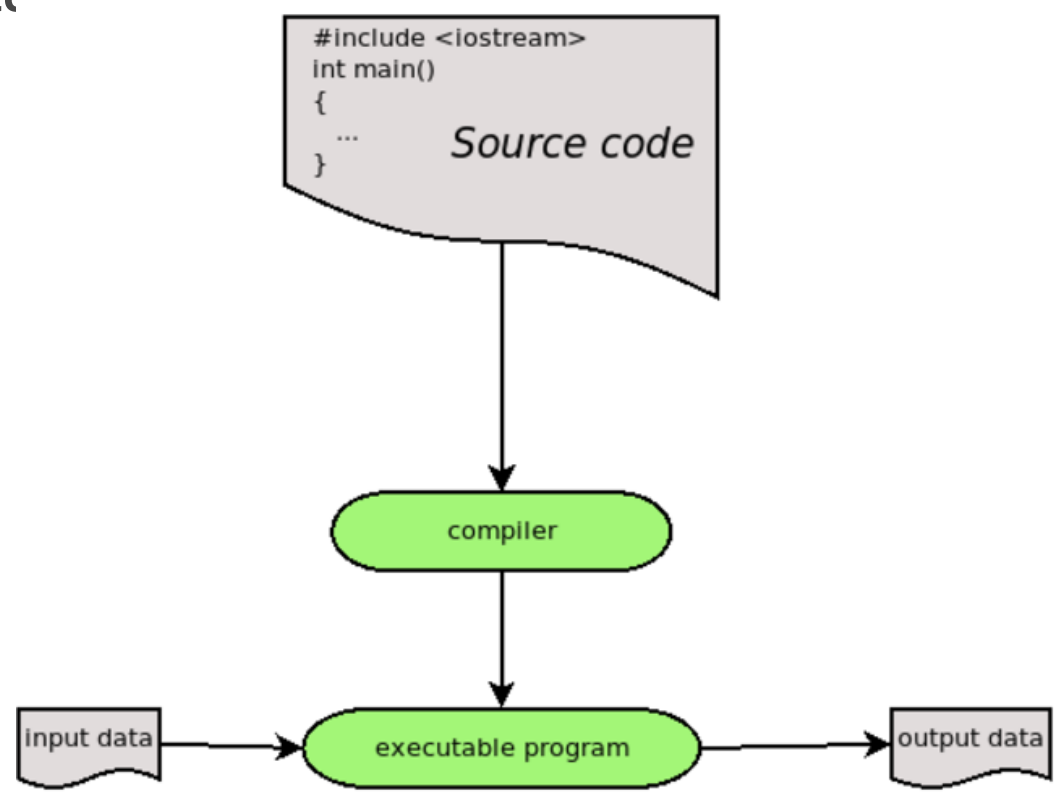




How is a program executed: Compilation

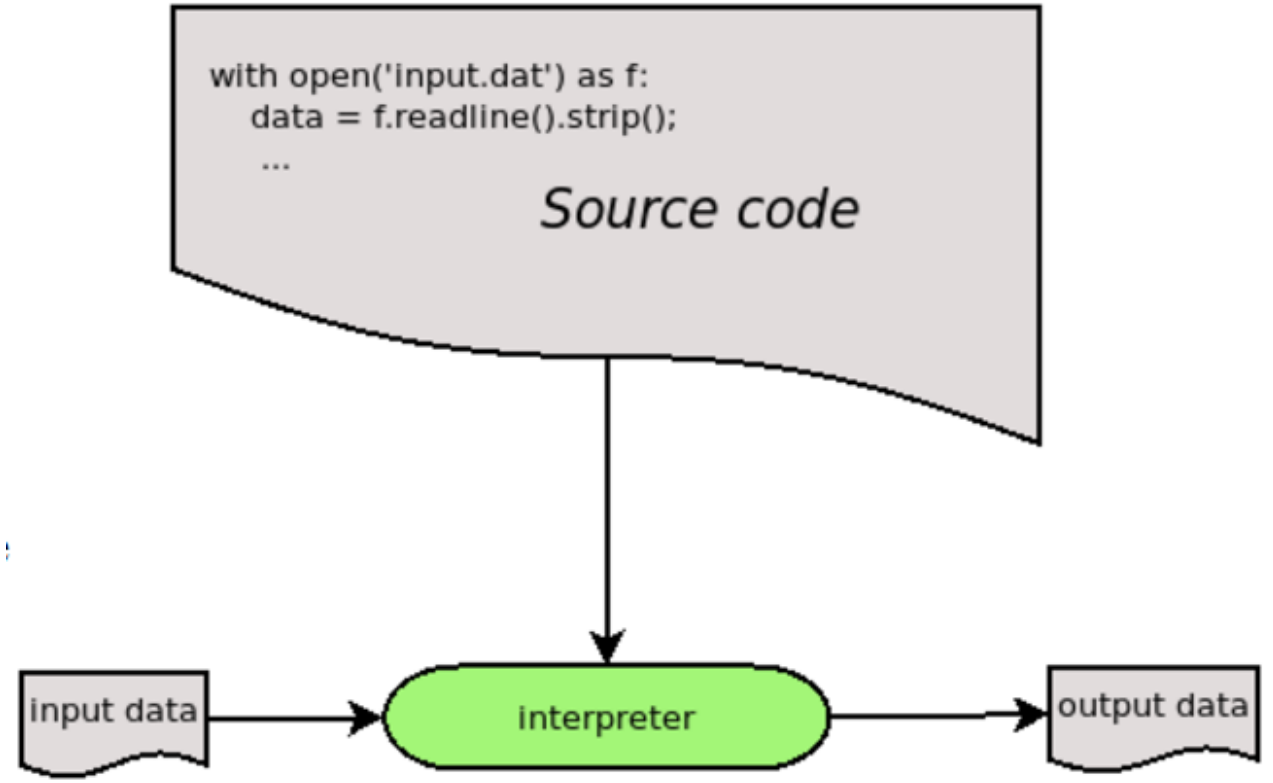
Compilation refers to the process by which human-readable source code written in any programming language is parsed then transformed into machine-executable binary code.

This transformation bridges the gap between the way we write code, which is in a form that's easy for humans to understand, and the way computers execute code, which is in binary form, composed of 0s and 1s.



Interpreted languages

An *interpreter* is an executable program that takes program source code (text) as input, parses it, translates it internally to determine what computation it describes, then simulates the execution of that program to process input data and generate output according to whatever we wrote our program to do.



What is a parser generator?

- A parser generator (such as ANTLR), accepts a grammar definition and generates a parser program

```
parser grammar ExprParser;
options { tokenVocab=ExprLexer; }

program
    : stat EOF
    | def EOF
    ;

stat: ID '=' expr ';'
    | expr ';'
    ;

def : ID '(' ID (',' ID)* ')' '{' stat* '}' ;

expr: ID
    | INT
    | func
    | 'not' expr
    | expr 'and' expr
    | expr 'or' expr
    ;

func : ID '(' expr (',' expr)* ')' ;
```

References:

https://www.antlr.org/
http://lab.antlr.org/

https://tomassetti.me
https://tomassetti.me/antlr-mega-tutorial/

4. The construction environment (Tools)

Why we need version control?

- Version control systems allow to keep all the **historical versions** of your software for easy tracking. It also benefits **team collaboration** and improves team efficiency. In addition, it can be used as a code repository, making build **automation and continuous integration** possible.
- Git is a **distributed version control software** (DCSV).
- **Github** is a git hosting platform that provides a cloud-based git repository for software projects.

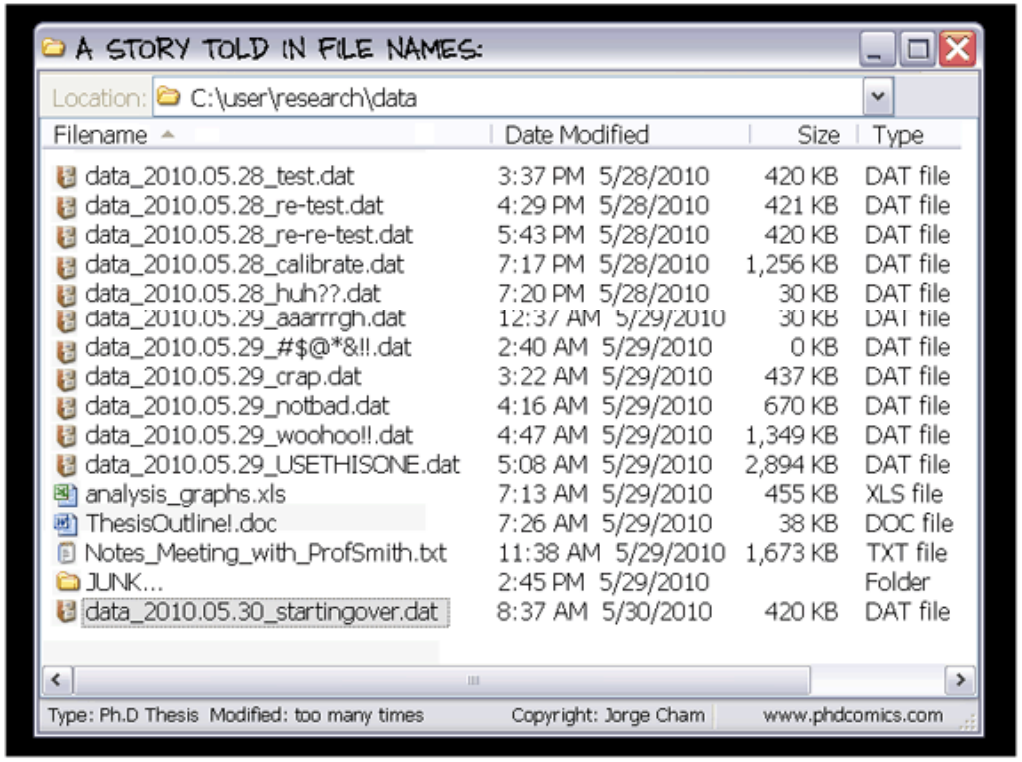


Fig. Versions nightmare

What is **source control** or **version control** or **revision control**

- Source control (also called revision control or version control) is a software management technique (not **project management** but **source code management**).
- Source control consists of different practices and techniques that tracks and provides control over changes to source code.
- Source control is needed:
  - When we have different versions of the same product to maintain.
  - When each part of the team works on their feature separately.
  - When we want to try a solution for a problem and we want to make sure we can go back to our last working version if things go wrong.

Source Control Operations

Different source control systems use different terminologies. All of them implement at least two operations:

- **Get**: retrieve a specific set of changes (files, directories, etc) from the repository.
- **Commit**: add a specific set of changes to the repository

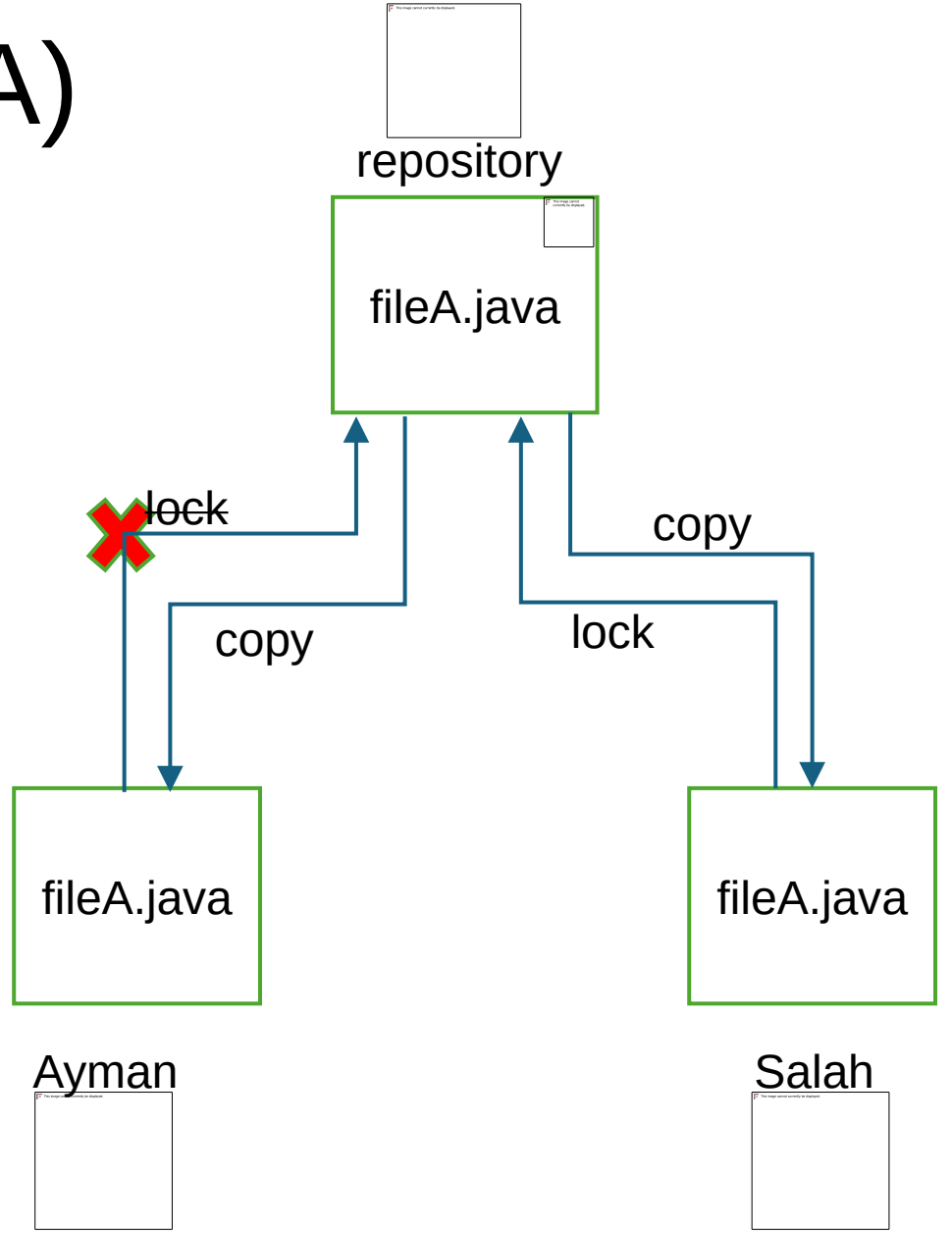
Some source control systems also provide a **locking mechanism** to prevents overwriting some specific files.:

- **Check-out**: Locking a specific set of files and then Getting the latest revision of those files
- **Check-in**: Unlocking a specific set of locked files and then Committing changes against those files

Different philosophies (A)

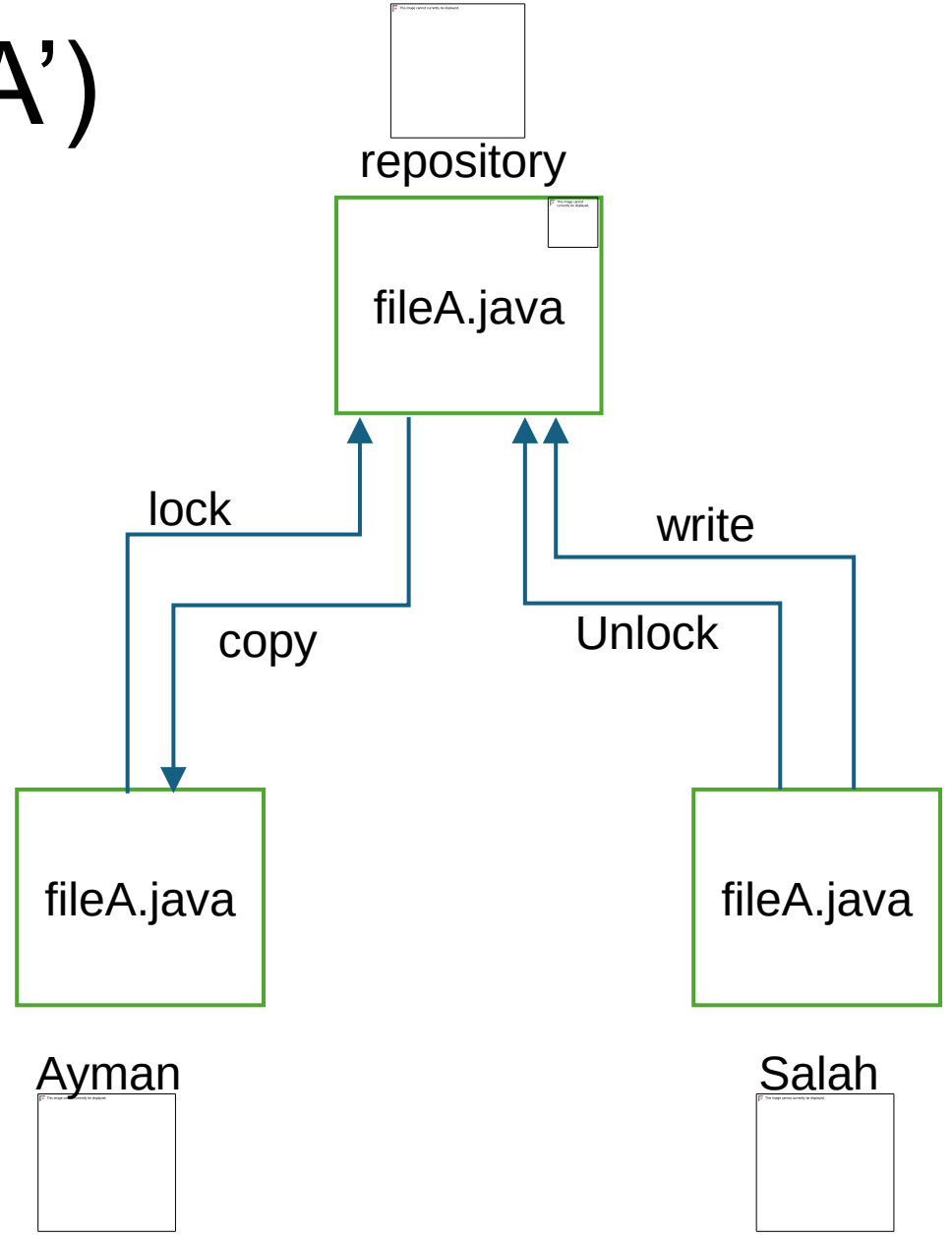
Revision control tools have two different philosophies, the first one is based on locking files (checkout) and the second is based on merging:

Check-out + Edit + Check-in – before making any changes to a file, a developer must check-out the file, and while it's checked-out, no other developers can make changes against it until the file is checked-in



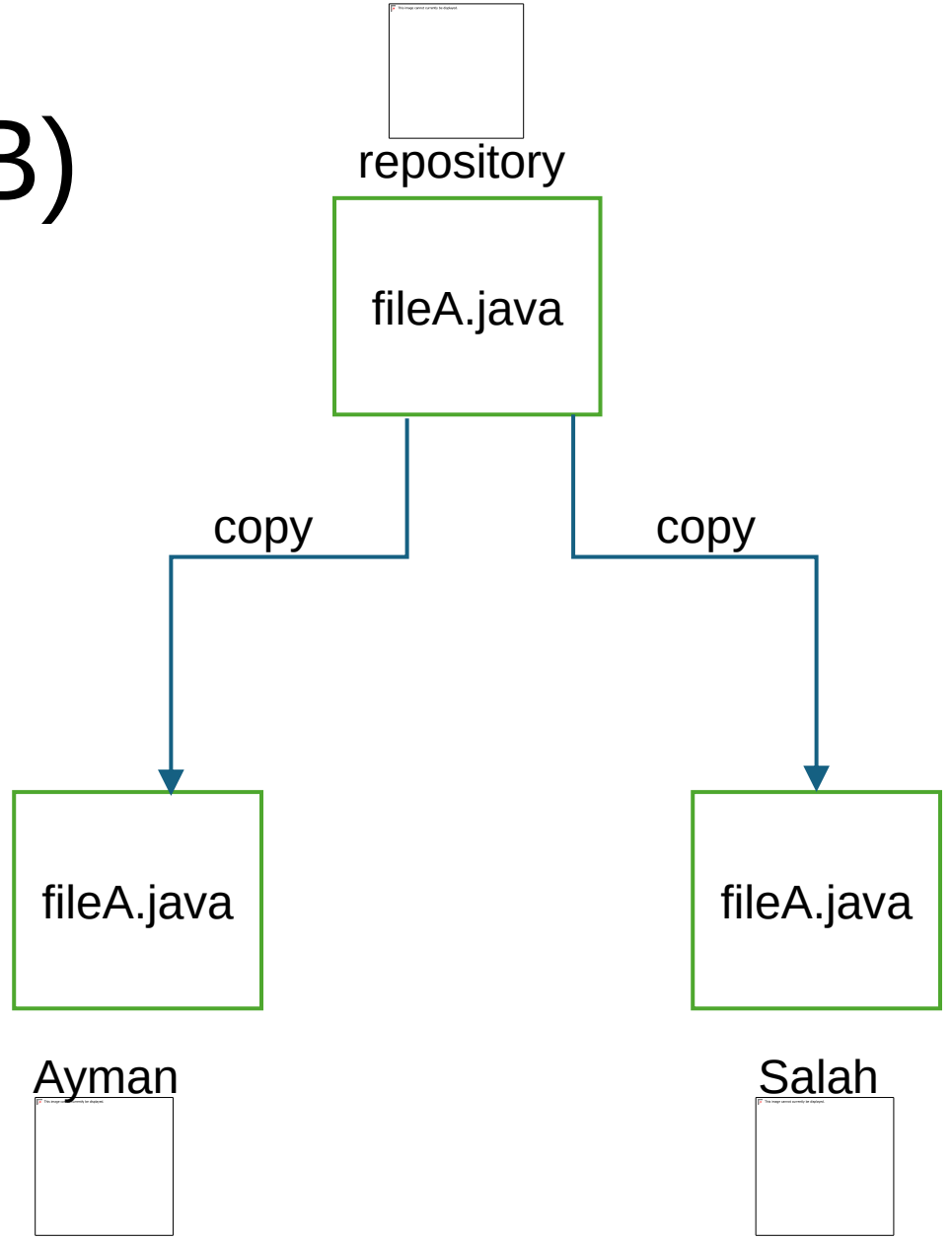
Different philosophies (A’)

- **Check-out + Edit + Check-in** – before making any changes to a file, a developer must check-out the file, and while it’s checked-out, no other developers can make changes against it until the file is checked-in



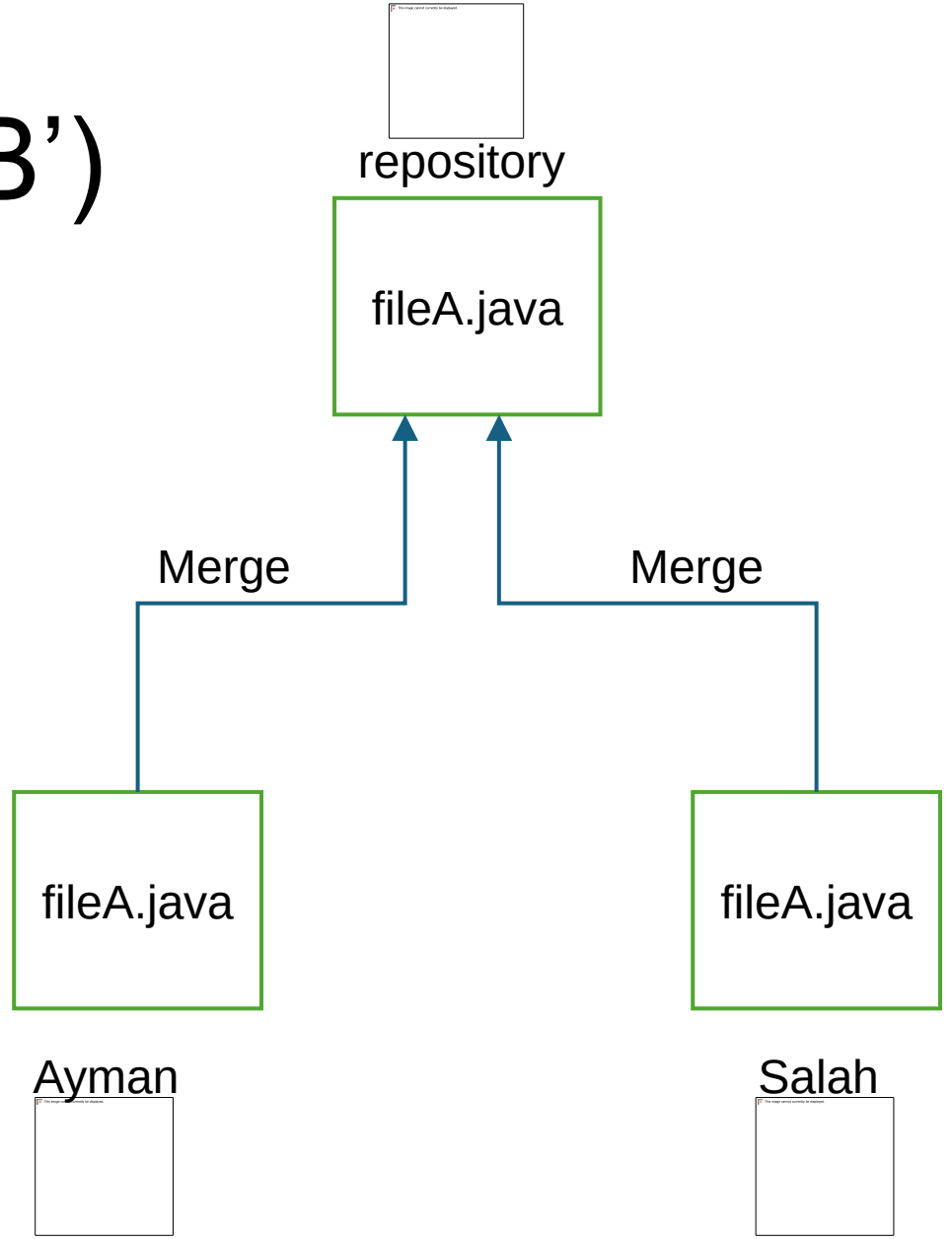
Different philosophies (B)

- **Copy + Modify + Merge:** before committing a change to the repository, a developer must get the latest version of the file and merge any changes made since the last time he retrieved it



Different philosophies (B’)

- **Copy + Modify + Merge:** before committing a change to the repository, a developer must get the latest version of the file and merge any changes made since the last time he retrieved it

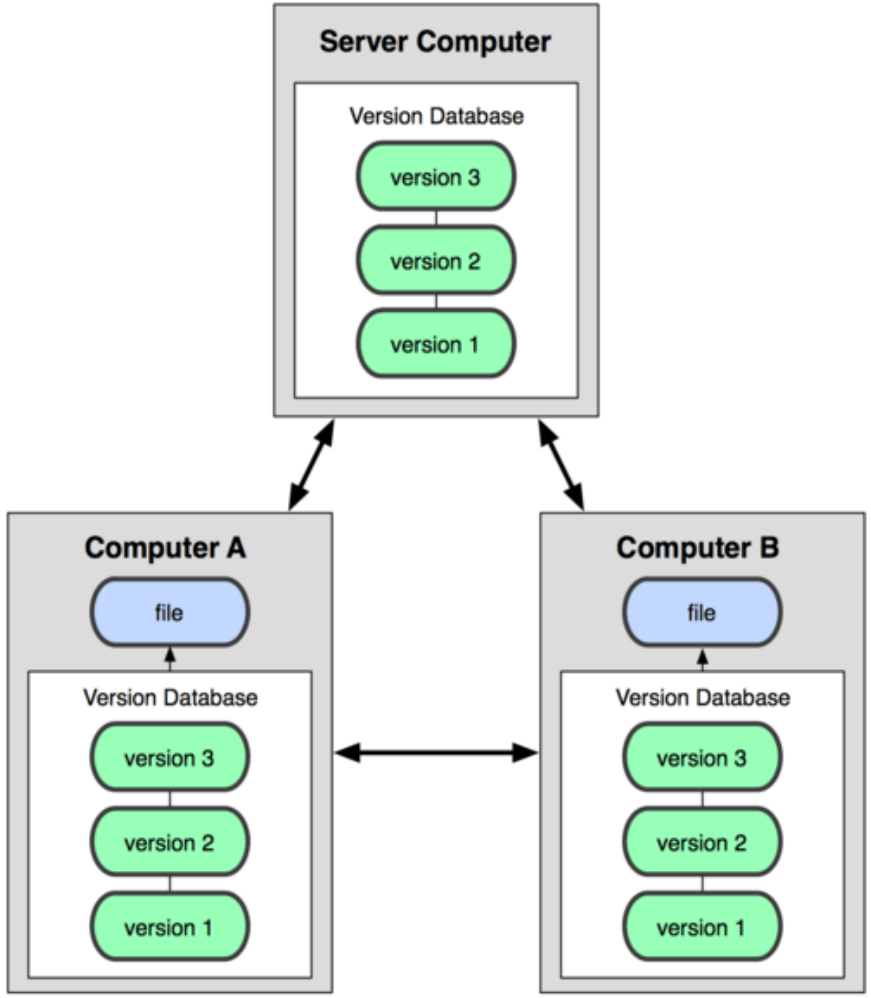


g

Distributed Version Control

A “remote master” repository is typically hosted on a server.

- Clones of the remote repository are maintained on each user’s computer
- If the server goes down, users can continue sharing code (provided they can connect over a network) by synch’ing to each others’ repositories
- If the network is unavailable, users can continue reading & writing to their own local repository – synch’ing with others later

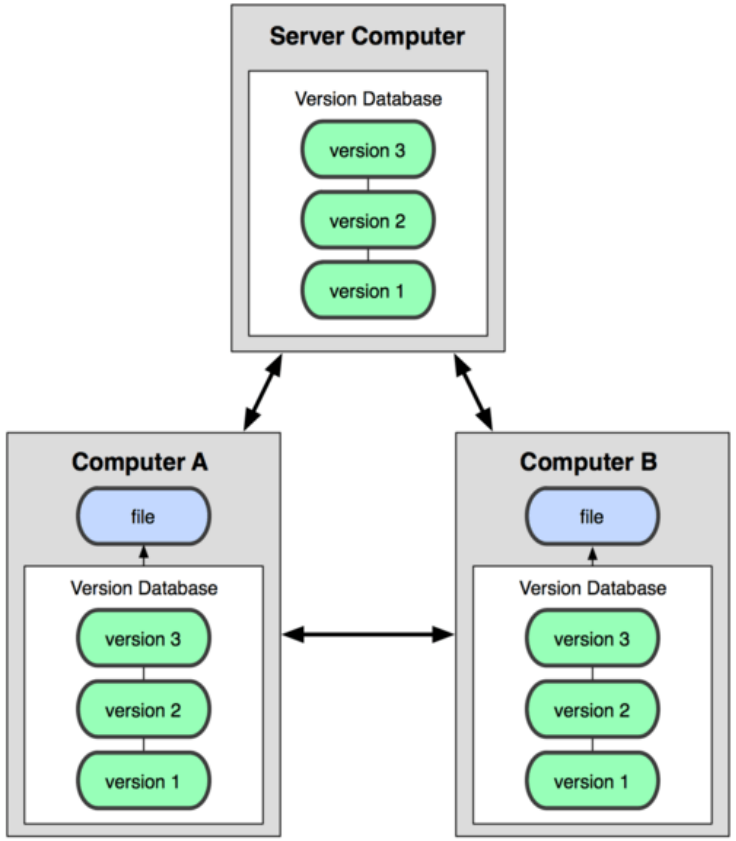


Git and Mercurial use this model

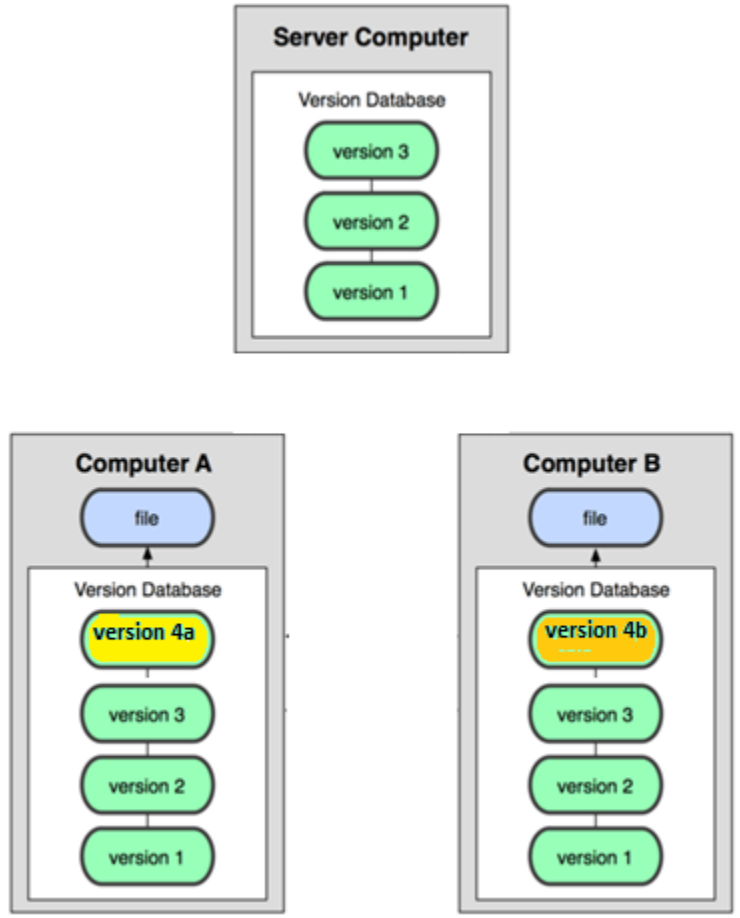
Image: http://git-scm.com/book/

- **git clone** – Downloads a GitHub repository to your computer.
- **git status** – Shows the current changes and repository status.
- **git add** – Selects changes to be included in the next commit.
- **git commit** – Saves the selected changes as a new version in your local repository.
- **git push** – Uploads your local commits to the remote GitHub repository.
- **git pull** – Downloads and integrates the latest changes from the remote repository.
- **git fetch** – Downloads remote changes without automatically merging them.
- **git branch** – Creates or lists branches for separate development.
- **git merge** – Combines changes from one branch into another.

Merging in a DVCS, part 1

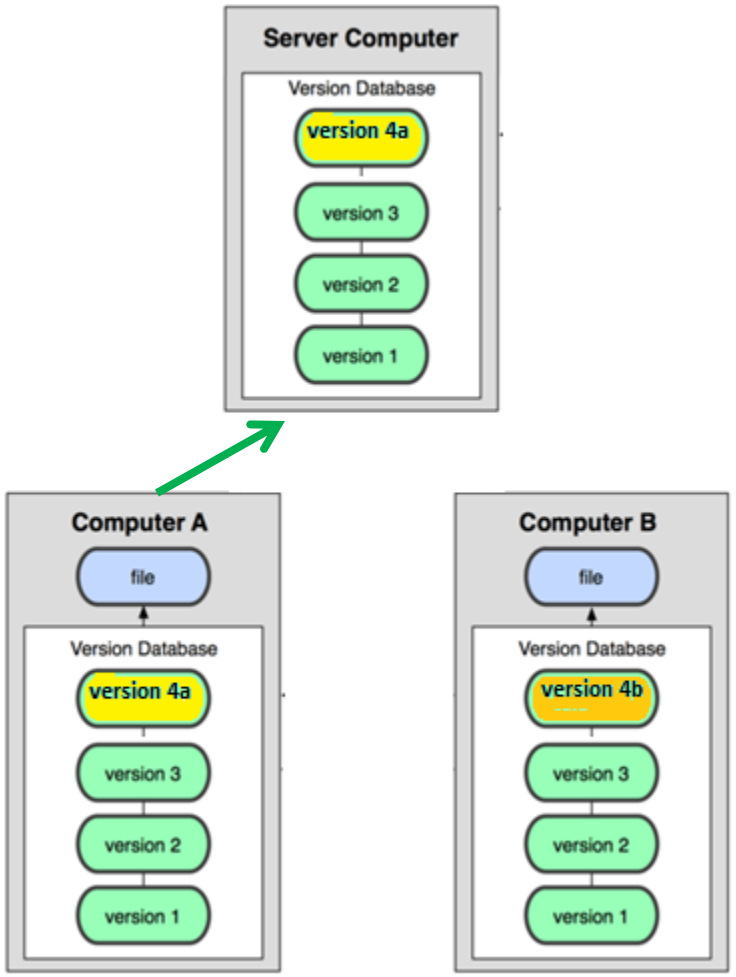


1. Initially, both users' repositories are synch'd to the remote master; All files are identical.

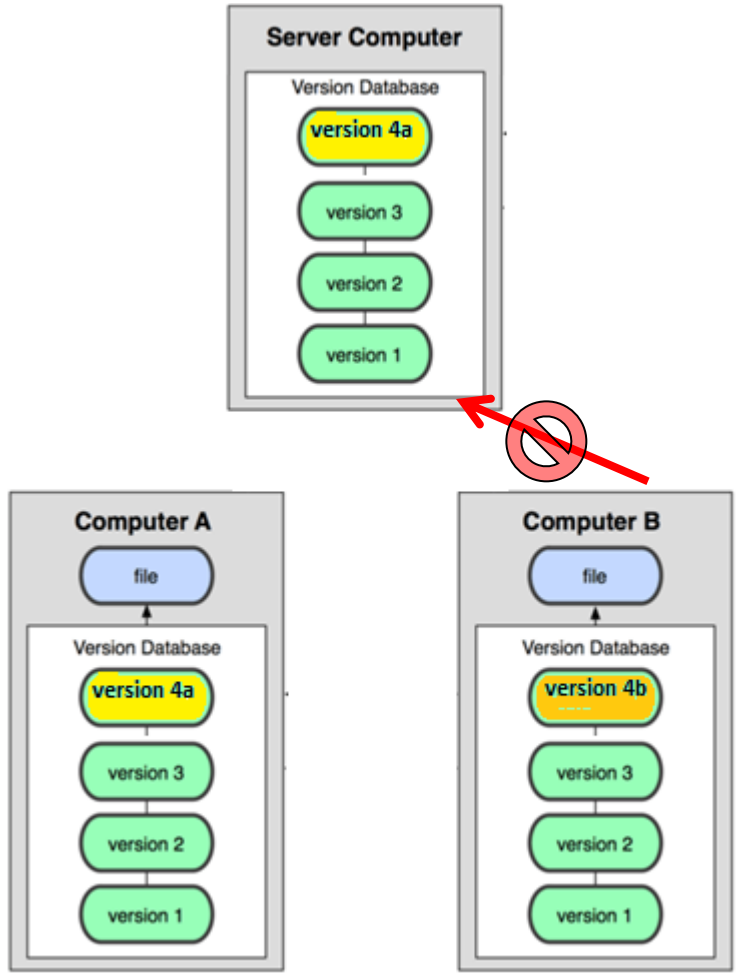


2. Both users edit the same file, and their local repositories reflect their changes

Merging in a DVCS, part 2

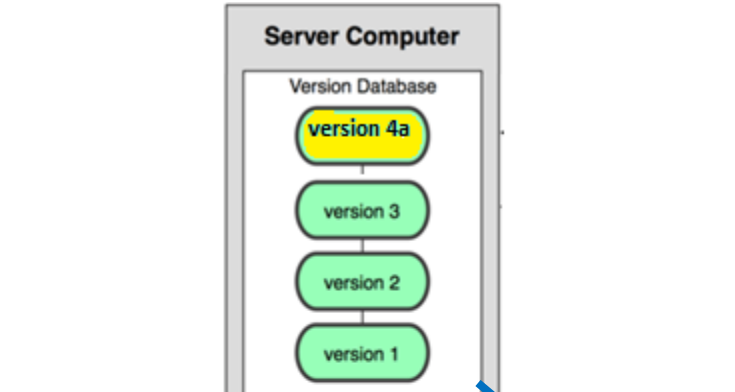


3. User A "pushes" his repository to the remote master to sync; this succeeds.

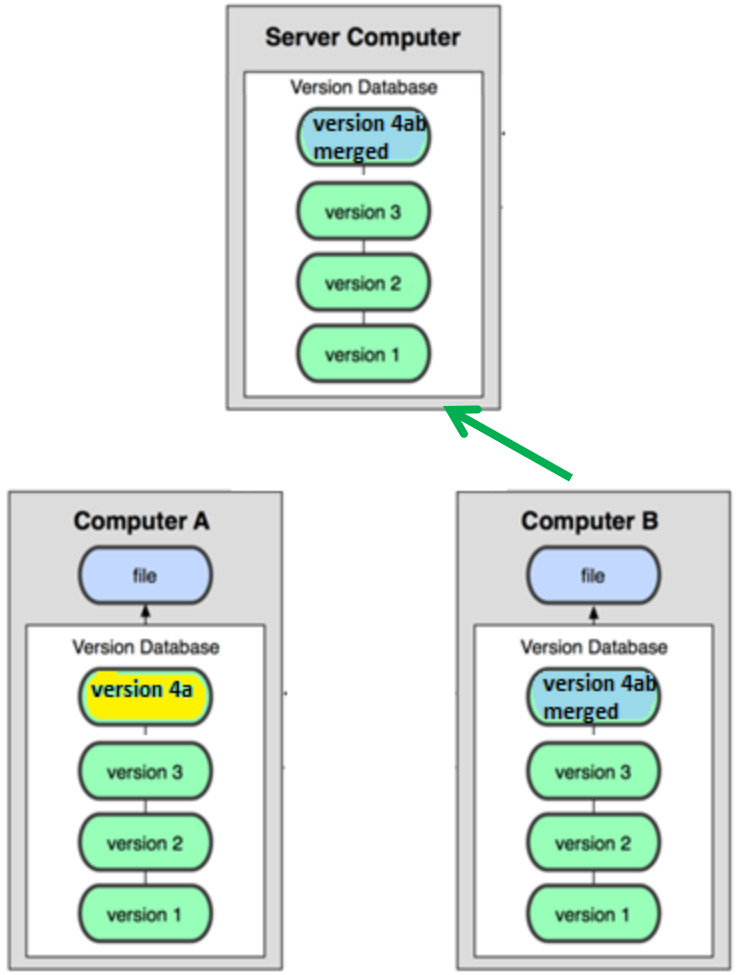


4. User B attempts to "push" his repository to the remote to sync; this fails due to User A's earlier push. Remote does not change.

Merging in a DVCS, part 3

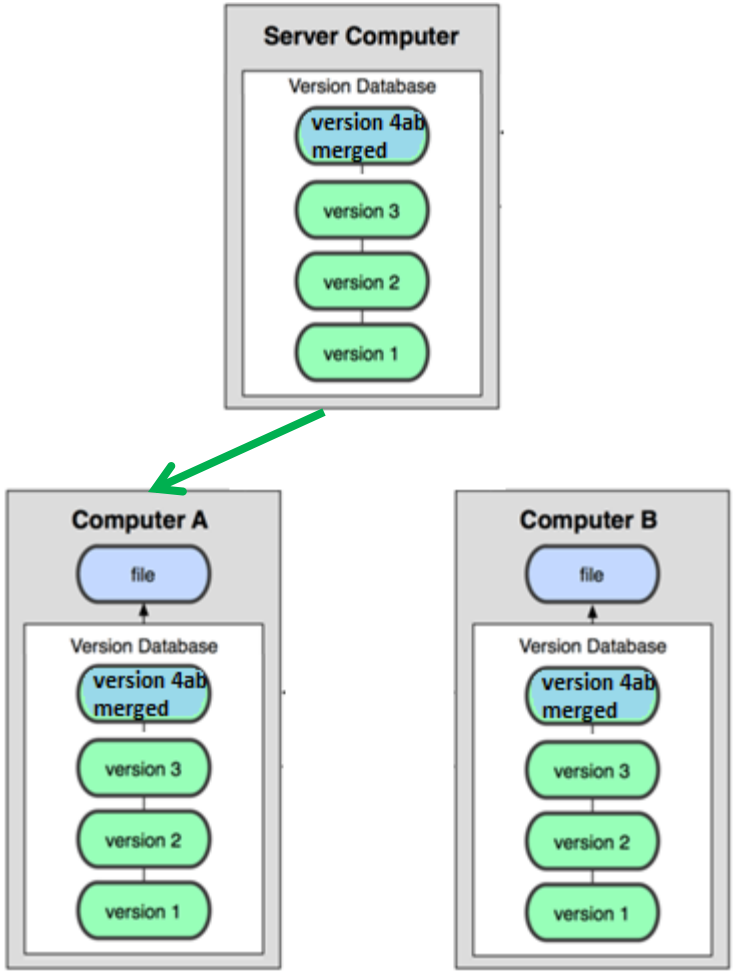


5. User B "pulls" User A's updates from the remote master, merging A and B together. Merging is automatic with some exceptions.



6. User B "pushes" his repository to the remote to synch; this now succeeds

Merging in a DVCS, part 4



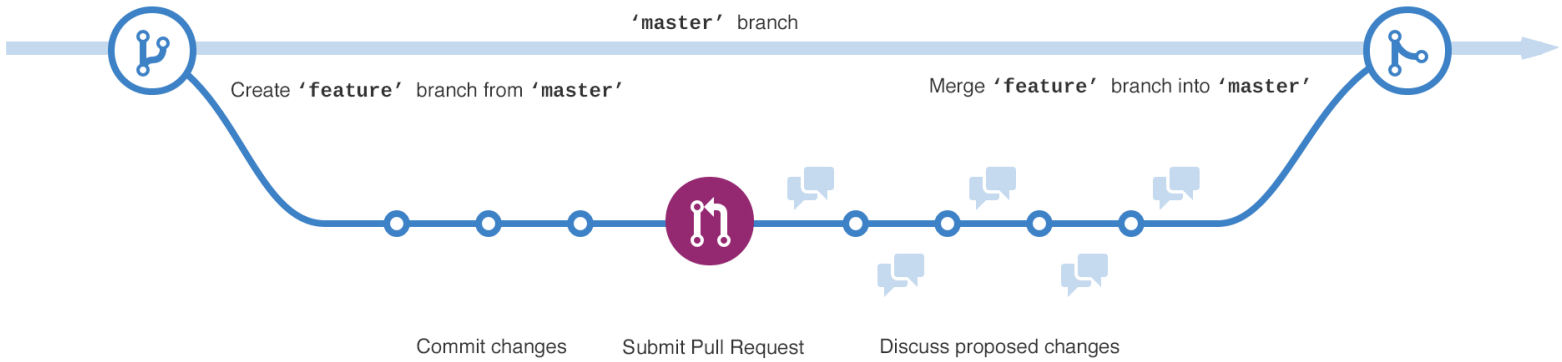
7. User B "pulls" from the remote, and everyone is in synch again.

Branching

Branching is the way to work on different versions of a repository at the same time.
By default your repository has one branch named ***Master or Main***. Branches are used to experiment and make edits before committing them to ***Master***.
When you create a branch off the ***Master*** , you're making a copy, or snapshot, of ***Master*** as it was at that point in time. If someone else made changes to the ***Master*** branch while you were working on your branch, you could **pull** in those updates.

This diagram shows:

- The **Master** branch
- A new branch called ***feature***.
- The journey that ***feature*** takes before it's **merged** into ***Master.***

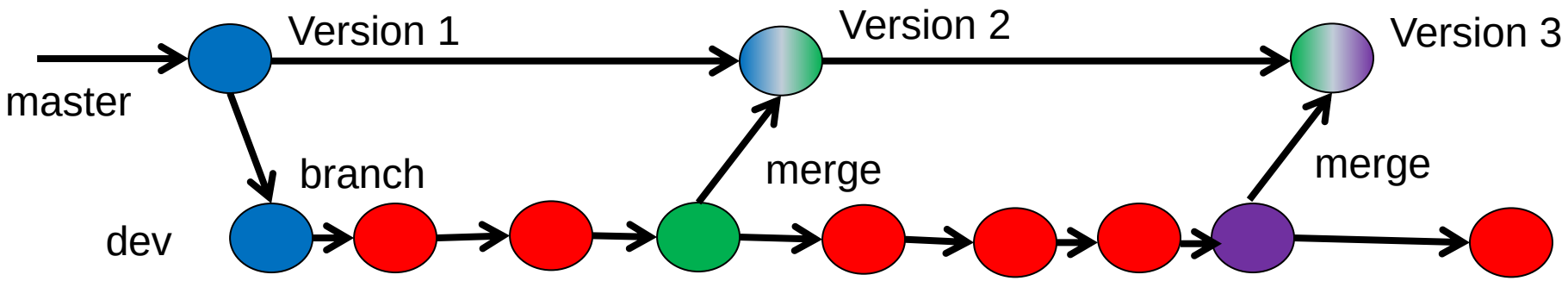


Developers use branches for keeping bug fixes and feature work separate from the production branch.
When a change is ready, they merge their branch into ***Master***.

Branching

The master branch should only be used for maintaining production-ready code (and related files)

- Code modified or created during a development cycle should be maintained on separate branches
- A second standard branch – often named "**dev**" is typically used to maintain code that is being actively worked on
  - At the beginning of a cycle, the **dev** branch is identical to master
  - During development, the **dev** branch maintains the evolving code
  - At the end of a cycle, the **dev** branch contains the completed code.
    - The **dev** branch is then merged into the **master** branch, and the process repeats with the next cycle (more on this later).



Git branching Strategy (GitFlow)

- Therefore, teams need to have a process in place to implement multiple changes at once. This is where having an efficient branching strategy becomes a priority for these teams.
- Branches are primarily used as a means for teams to develop features giving them a separate workspace for their code. These branches are usually merged back to a master branch upon completion of work. In this way, features (and any bug and bug fixes) are kept apart from each other allowing you to fix mistakes more easily.
- This means that branches protect the mainline of code and any changes made to any given branch don't affect other developers.

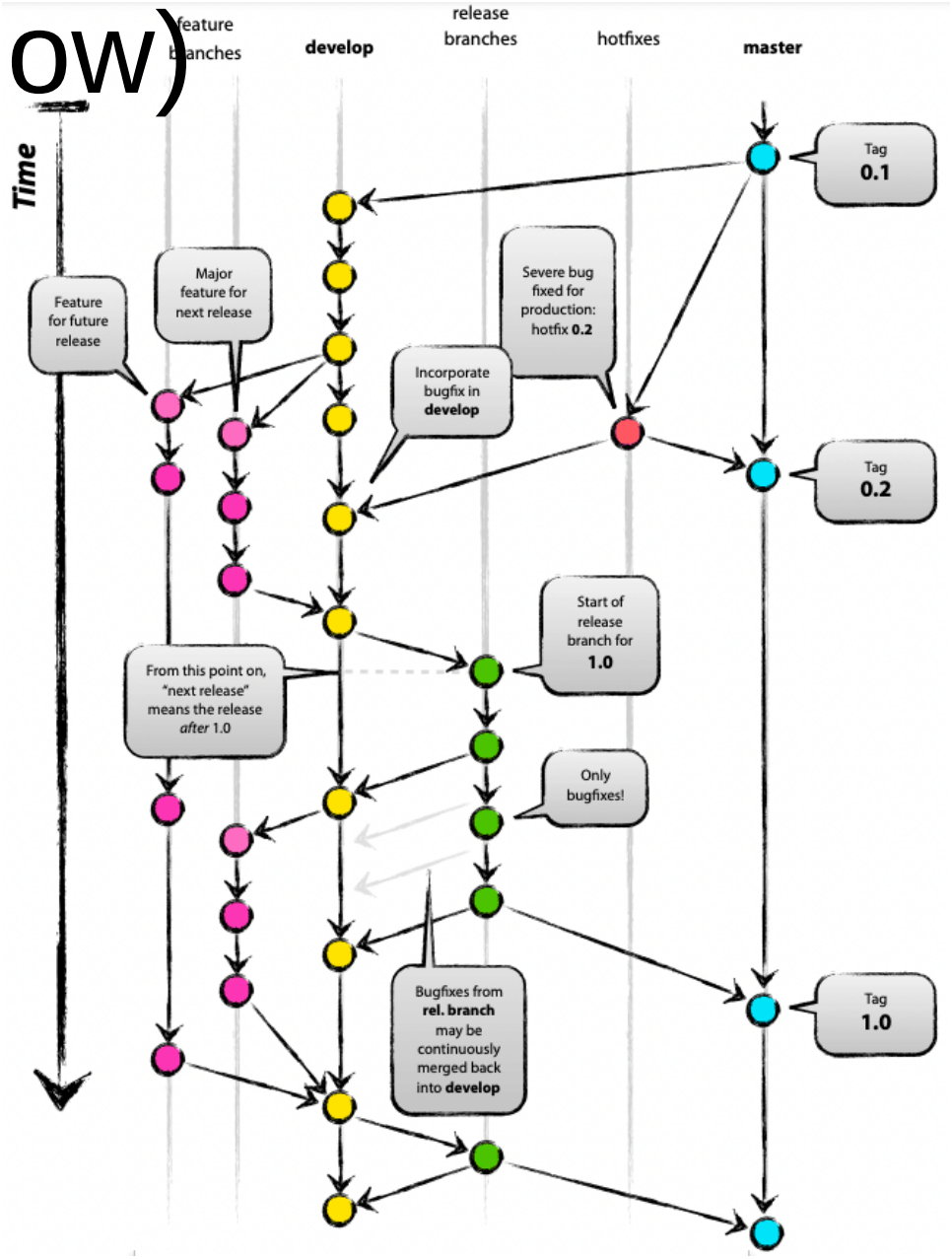
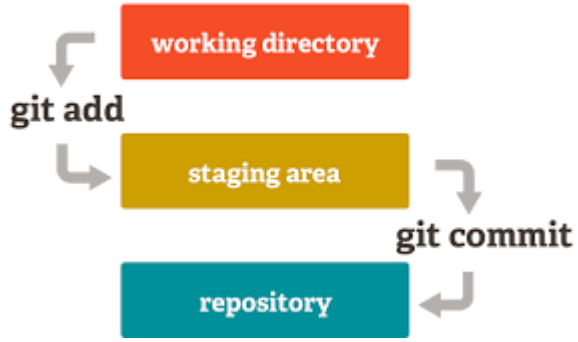


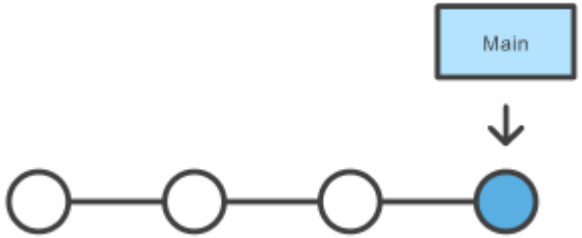
Fig. GitFlow branching strategy

Reading: https://www.abtasty.com/blog/git-branching-strategies/

Git commit

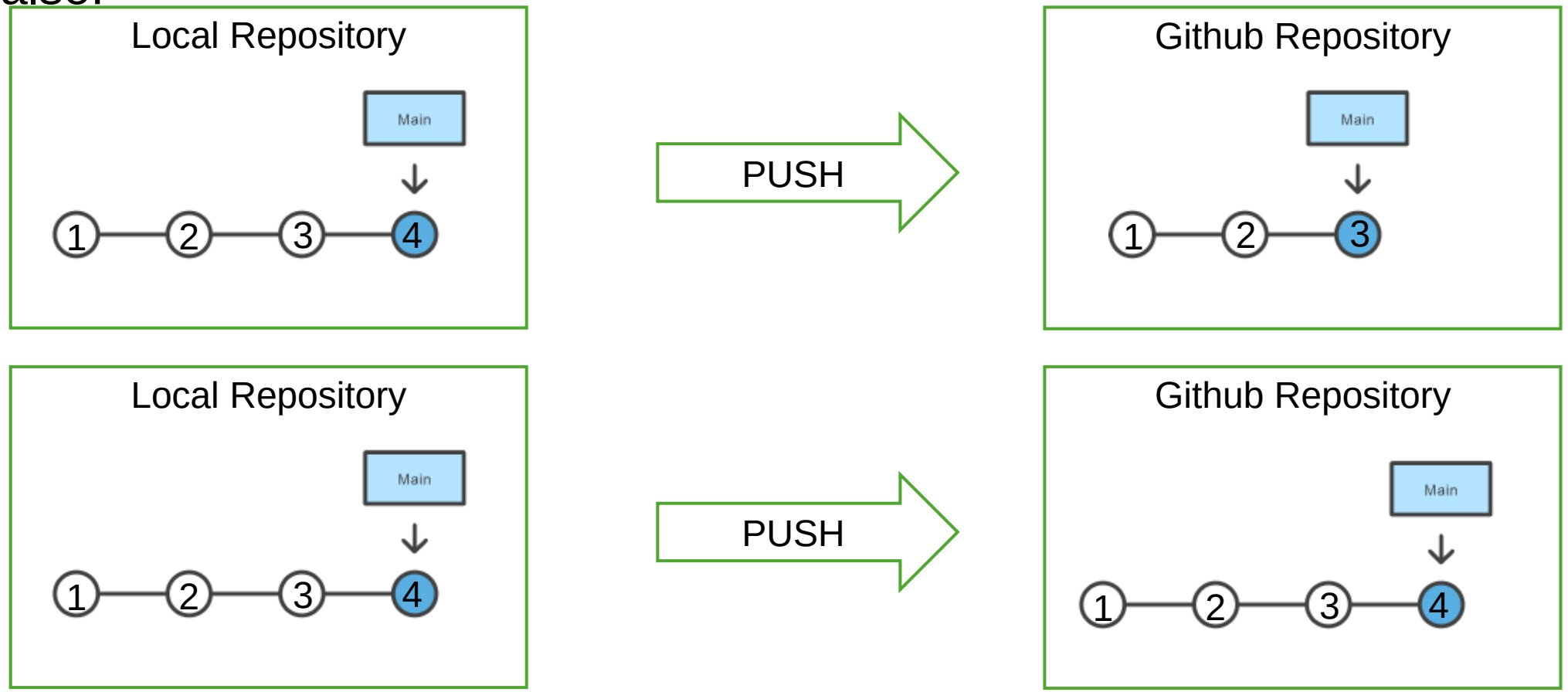
- To add code to git, it has to be added to the “staging area”. It is a way to tell git which files have to be added to the repository (or committed).
- We do this by using “git add file_name” or “git add .” to add all changes made in the staging area.
- Then we “commit”, which means that we create a new version of the code





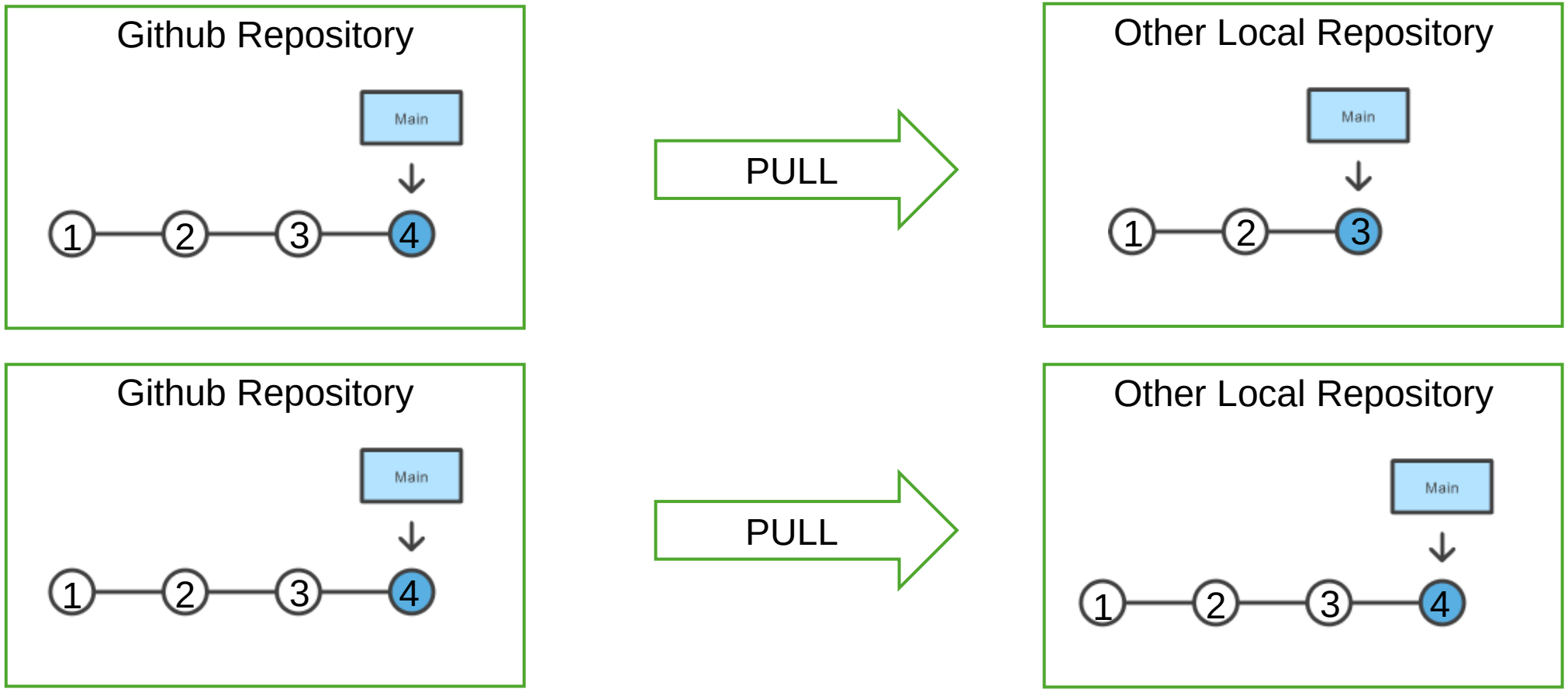
Git PUSH

- Once we create a commit on our local repository, if we have the necessary privileges on the github repository, we can “push” the changes to the shared repository on github so that our teammates can have the new code also:



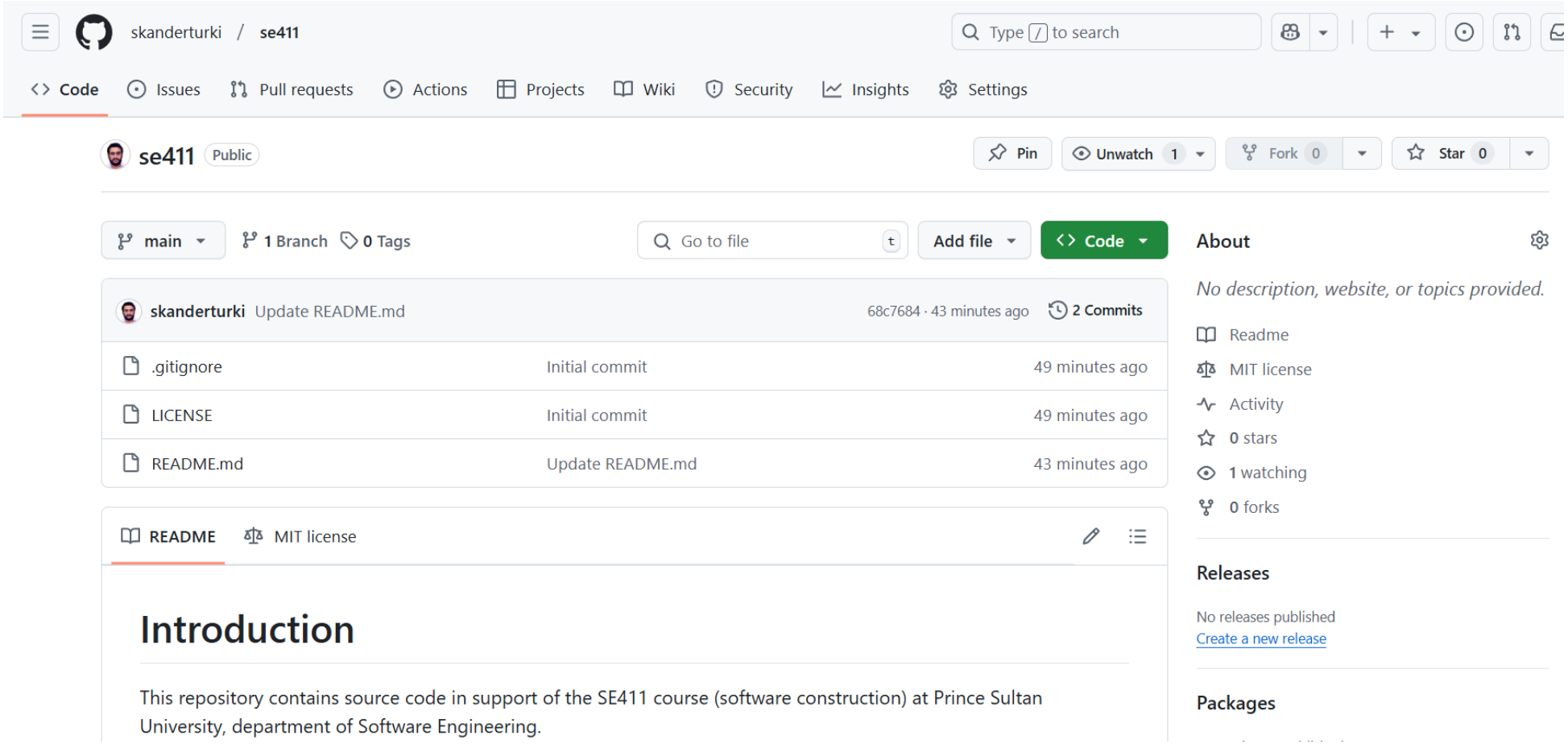
Git PULL

- Our teammates, now have to “PULL” the new changes to their local repository to have the most recent version:



GitHub: An online solution

- GitHub is a code hosting platform for version control and collaboration. It lets you and others work together on projects from anywhere.



Collaborative Construction: Code Reviews

- Most construction defects are found by people reading code, not by running it.
- Code review = a teammate reads your change before it merges.
- **Benefits:**
  - Catches bugs early and spreads knowledge across the team.
  - Improves readability and enforces shared standards.
- Review small changes often; be specific and focus on the code.
- **Modern twist: AI can pre-review a diff, but a human still owns the decision.**

Git + AI: Reviewable Diffs, Safe Commits

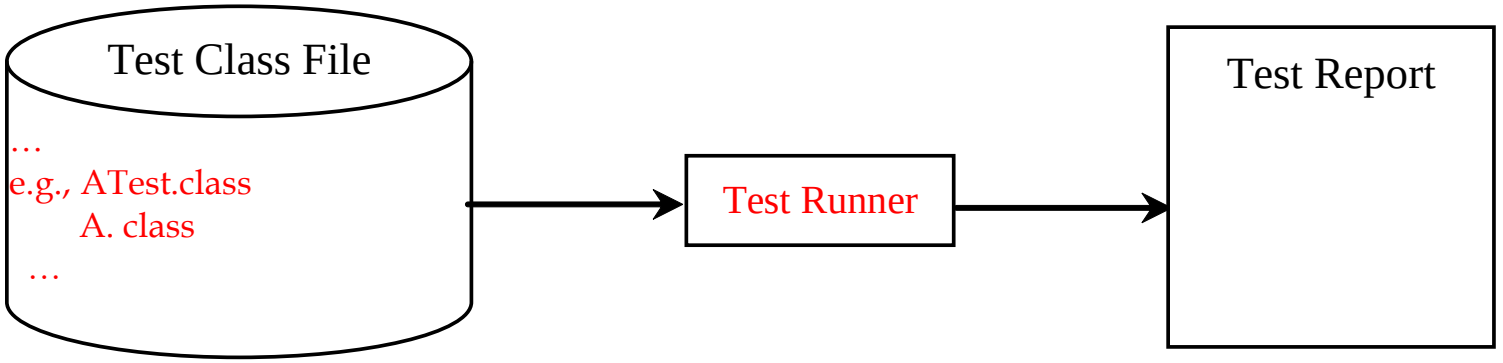
- AI fits a Git workflow naturally, with discipline:
  - Let AI draft commit messages, then check they match the diff.
  - Keep AI-generated changes in small, reviewable commits.
  - Review every AI diff before you stage it.
- **Two safety rules:**
  - Never commit secrets an assistant may echo back (keys, passwords).
  - Add AI tool artifacts and local config to .gitignore.

4.2. Verification with JUnit

JUnit Basics

JUnit is the de facto framework for unit testing Java programs. JUnit is a third-party open source library packed in a jar file. The jar file contains a tool called *test runner*, which is used to run test programs.

Suppose you have a class named A. To test this class, you write a test class named ATest. This test class, contains the methods you write for testing class A. The test runner executes ATest to generate a test report, as shown in Figure 44.1.



Developer Testing & Test-First (TDD)

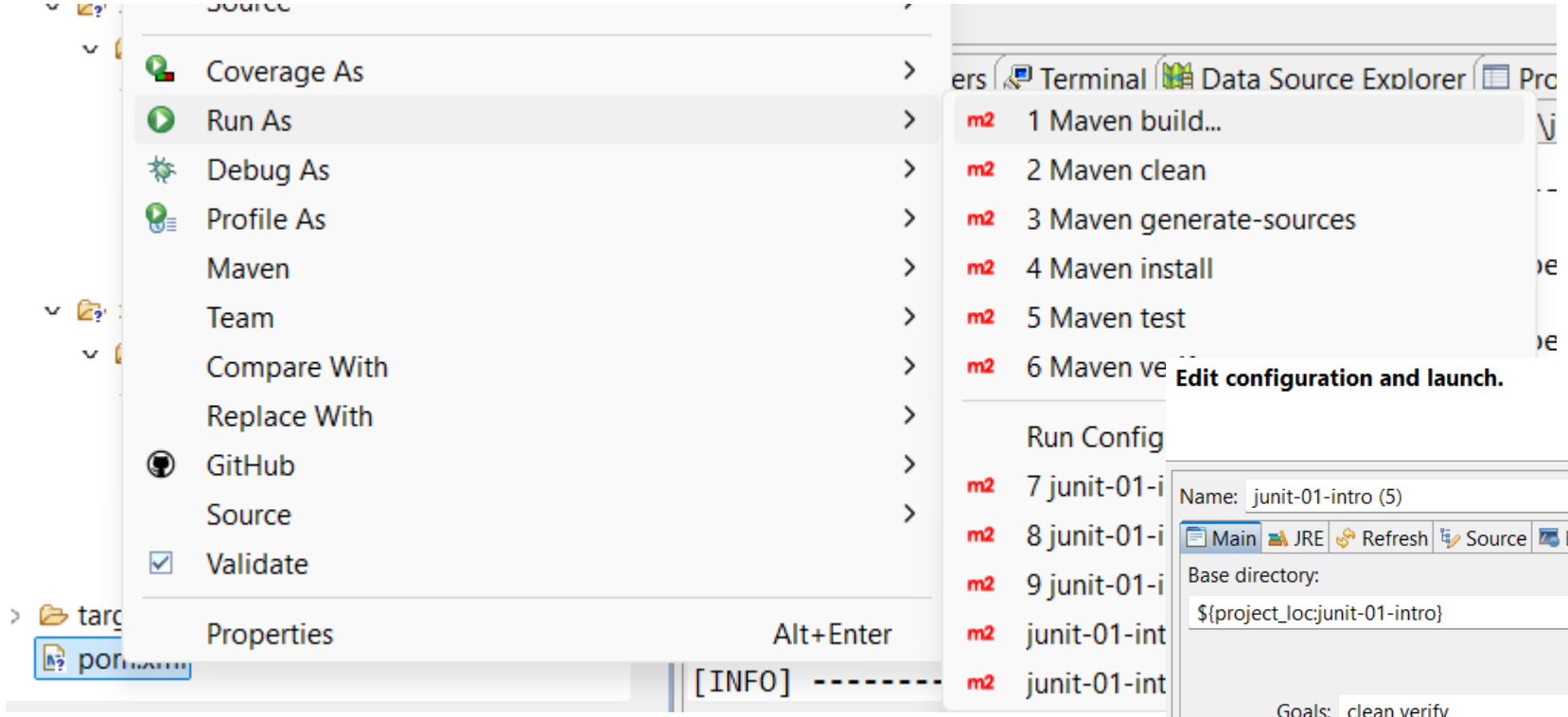
- Developers test their own code, testing is part of construction, not a later phase.
- **Test-First / TDD loop:**
  - Red: write a failing test for the next small behavior.
  - Green: write the simplest code that makes it pass.
  - Refactor: clean up with the test as a safety net.
- Benefits: better design, instant feedback, a growing regression suite.
- **JUnit (next slides) is the tool we use to do this in Java.**

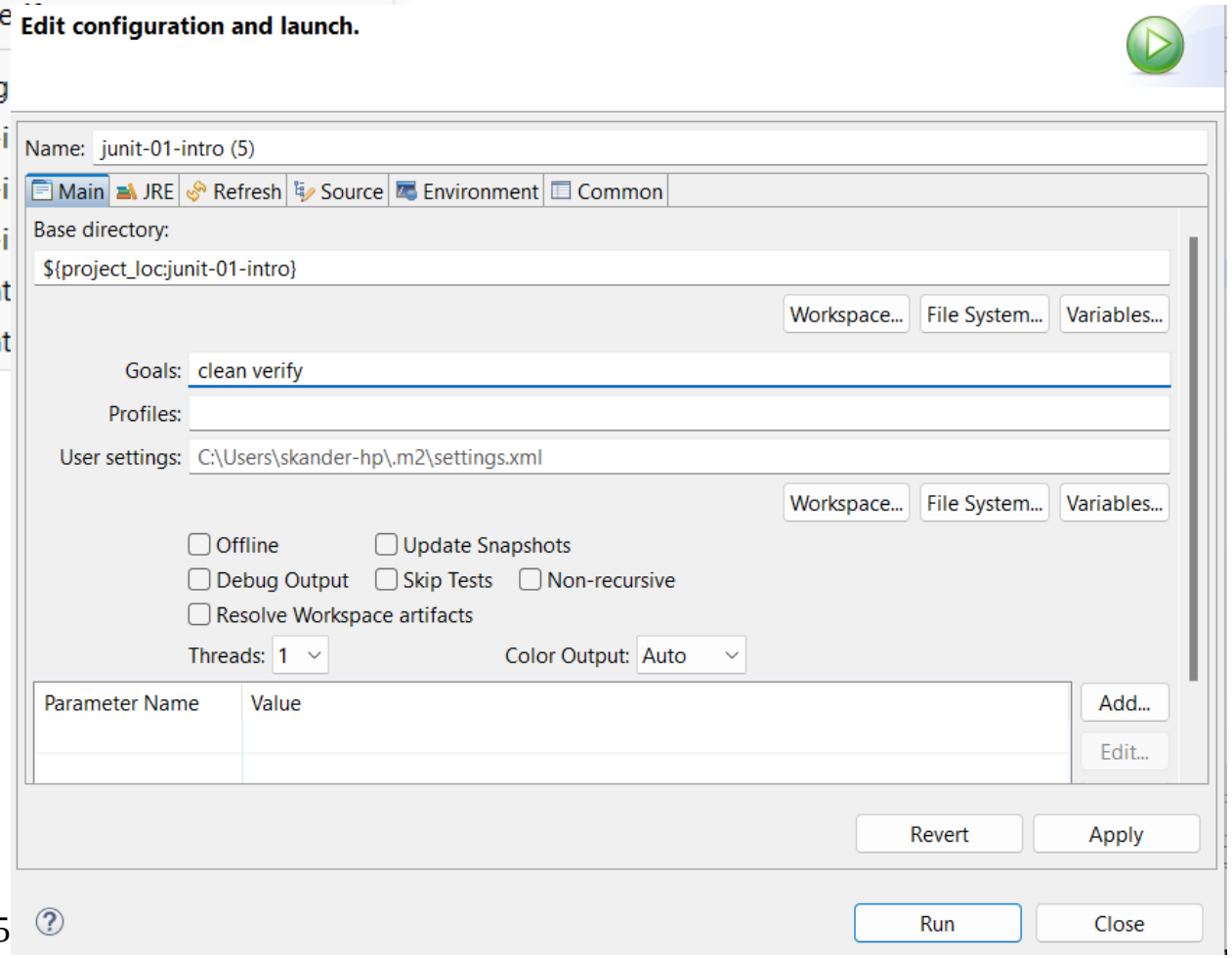
A JUnit Test Class

To use JUnit, create a test class. By convention, if the class to be tested is named A, the test class should be named ATest.
A simple template of a test class may look like this:

```
package mytest;
import org.junit.*;
import static org.junit.Assert.*;
public class ATest {
  @Test
  public void m1() {
    // Write a test method
  }
  @Test
  public void m2() {
    // Write another test method
  }
  @Before
  public void setUp() throws Exception {
    // Common objects used by test methods may be set up here
  }
}
```

Run the Test





To run the test, right-click on the pom.xml
and select Maven Build.
Then specify this Maven goal: *clean verify*

Test ArrayList

Example of a test class for testing java.util.ArrayList.

```
package mytest;
import org.junit.*;
import static org.junit.Assert.*;
import java.util.*;

public class ArrayListTest {
  private ArrayList<String> list = new ArrayList<String>();

  @Before
  public void setUp() throws Exception {
  }

  @Test
  public void testInsertion() {
    list.add("Beijing");
    assertEquals("Beijing", list.get(0));
    list.add("Shanghai");
    list.add("Hongkong");
    assertEquals("Hongkong", list.get(list.size() - 1));
  }

  @Test
  public void testDeletion() {
    list.clear();
    assertTrue(list.isEmpty());

    list.add("A");
    list.add("B");
    list.add("C");
    list.remove("B");
    assertEquals(2, list.size());
  }
}
```

Test the Multiply method

```java
public class Util {

  public int multiply(int a, int b) {
    return a * b;
  }
}
```



```java
public class UtilTest {

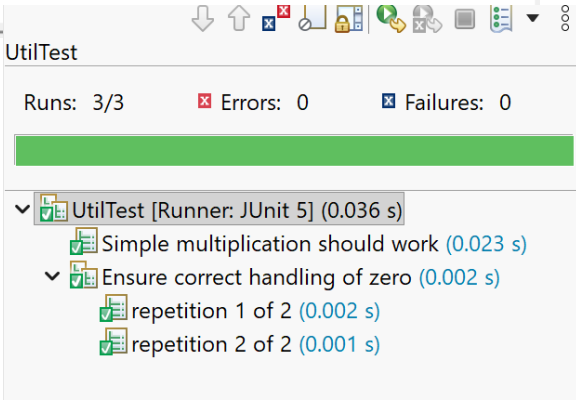
  private Util calculator;

  @BeforeEach
  void setUp() throws Exception {
    calculator = new Util();
  }

  @Test
  @DisplayName("Simple multiplication should work")
  void testMultiply() {
    assertEquals(20, calculator.multiply(4,5), "Regular multiplication should work")
  }

  @RepeatedTest(2)
  @DisplayName("Ensure correct handling of zero")
  void testMultiplyWithZero() {
    assertEquals(0, calculator.multiply(0,5), "Multiple with zero should be zero");
    assertEquals(0, calculator.multiply(5,0), "Multiple with zero should be zero");
  }
}
```

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running edu.psu.junit_01_intro.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.033 s -- in edu.psu.junit_01_intro.AppT
[INFO] Running edu.psu.junit_01_intro.UtilTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in edu.psu.junit_01_intro.Util
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
```



Source code: junit-01-intro

Testing + AI: You Own the Tests

- AI can draft unit tests in seconds, but the tests are yours.
- **The danger:**
  - AI tests can encode the SAME wrong assumption as the code.
  - Passing AI tests on AI code proves very little.
- **Best practice:**
  - Treat the test as the specification you give the AI.
  - Add your own edge cases: null, empty, boundaries, errors.
- **Tests are the contract, AI writes code to it; you own it.**

Verify the AI #1: Trust, but Test

- An AI assistant produced this isValid(). It compiles and looks fine, find the bug with a test.

```
// Prompt: “valid = an even number that is
//         greater than OR EQUAL TO 10”

// AI-generated code:
public boolean isValid(int number) {
    return number > 10 && number % 2 == 0;
}

// Looks right... compiles... runs.
```



```
// The boundary test you must write:
@Test
void boundaryValueIsValid() {
    // 10 is even AND >= 10 -> valid
    assertTrue(v.isValid(10));
}

// FAILS: isValid(10) returns false
// Bug: “> 10” should be “>= 10”
```

Lesson: the code compiled, ran, and looked correct. Only a boundary test exposed the off-by-one. Always test the edges of AI output.

Exercise-1: Trust, but Test

Scenario: You ask an AI assistant to implement the following requirement:

- A student passes if their mark is **greater than or equal to 50**.
- The AI generates this code:

```
public boolean isPass(int mark)
{ return mark > 50; }
```

Your Task: Read the AI-generated code carefully. Do you see a potential problem? Write **JUnit tests** for the following values:

- 49
- 50
- 51
- Run the tests. Identify which test fails and explain **why**.
- Fix the AI-generated code so that all tests pass.

Solution:

- Junit Tests

```
@Test
void boundary49IsInvalid() {
assertFalse(isPass(49));  // 49 should NOT pass
}
@Test
void boundary50Isvalid() {
assertTrue(isPass(50));   // 50 SHOULD pass
}
@Test
void boundary51Isvalid() {
assertTrue(isPass(51));   // 51 SHOULD pass
}
```

- Fix Code:

```
public boolean isPass(int mark) {
    return mark >= 50;
    }
```

Exercise-2: Trust, but Test

- **Scenario: Scenario:** You ask an AI assistant to write a method that checks whether a username is valid.
- **Requirement:** A username is valid if it is **not empty and contains only letters**.
- The AI generates:

```
public boolean isValidUsername(String username) {
return username.matches("[a-zA-Z]+");
 }
```

Your Task: Read the AI-generated code carefully. Do you see a potential problem? Write **JUnit tests** for the following values:

- Asma" → valid
- "" → invalid
- "Asma123" → invalid
- "Asma Sattar" → invalid
- null → invalid
- Run the tests. Identify whether the AI-generated code handles **all edge cases correctly**. If a test fails, explain why and fix the code.

Solution

```
@Test
void validUsername() {
    assertTrue(isValidUsername("Asma"));
}
@Test
void emptyUsernameIsInvalid() {
    assertFalse(isValidUsername(""));
}
  @Test
void usernameWithNumbersIsInvalid() {
    assertFalse(isValidUsername("Asma123"));
}
```

```
@Test
void usernameWithSpaceIsInvalid() {
    assertFalse(isValidUsername("Asma Sattar"));
}

@Test
void nullUsernameIsInvalid() {
    assertFalse(isValidUsername(null));
}
```

Solution contd..

```
public boolean isValidUsername(String username) {
    if (username == null || username.isEmpty()) {
        return false;
    }

    return username.matches("[a-zA-Z]+");
}
```